serves as a kind of token—an authorization to mutate or access the cell—but it serves no other purpose.

The `STRef` trait is sealed, and the only way to construct an instance is by calling the `apply` method on the `STRef` companion object. The `STRef` is constructed with an initial value for the cell, of type `A`. But what's returned is not a naked `STRef`, but an `ST[S, STRef[S,A]]` action that constructs the `STRef` when run and given the token of type `S`. It's important to note that the `ST` action and the `STRef` that it creates are tagged with the *same* `S` type.

At this point, let's try writing a trivial `ST` program. It's awkward right now because we have to choose a type `S` arbitrarily. Here, we arbitrarily choose `Nothing`:

```
for {
  r1 <- STRef[Nothing,Int](1)
  r2 <- STRef[Nothing,Int](1)
  x  <- r1.read
  y  <- r2.read
  _  <- r1.write(y+1)
  _  <- r2.write(x+1)
  a  <- r1.read
  b  <- r2.read
} yield (a,b)
```

This little program allocates two mutable `Int` cells, swaps their contents, adds one to both, and then reads their new values. But we can't yet *run* this program because `run` is still protected (and we could never actually pass it a value of type `Nothing` anyway). Let's work on that.

### 14.2.3 Running mutable state actions

By now you may have figured out the plot with the `ST` monad. The plan is to use `ST` to build up a computation that, when run, allocates some local mutable state, proceeds to mutate it to accomplish some task, and then discards the mutable state. The whole computation is referentially transparent because all the mutable state is private and locally scoped. But we want to be able to *guarantee* that. For example, an `STRef` contains a mutable `var`, and we want Scala's type system to guarantee that we can never extract an `STRef` out of an `ST` action. That would violate the invariant that the mutable reference is local to the `ST` action, breaking referential transparency in the process.

So how do we safely run `ST` actions? First we must differentiate between actions that are safe to run and ones that aren't. Spot the difference between these types:

- `ST[S, STRef[S, Int]]` (not safe to run)
- `ST[S, Int]` (completely safe to run)

The former is an `ST` action that returns a mutable reference. But the latter is different. A value of type `ST[S,Int]` is literally just an `Int`, even though computing the `Int` may involve some local mutable state. There's an exploitable difference between these two types. The `STRef` involves the type `S`, but `Int` doesn't.

We want to disallow running an action of type `ST[S, STRef[S,A]]` because that would expose the `STRef`. And in general we want to disallow running any `ST[S,T]`

where `T` involves the type `S`. On the other hand, it's easy to see that it should always be safe to run an `ST` action that doesn't expose a mutable object. If we have such a pure action of a type like `ST[S,Int]`, it should be safe to pass it an `S` to get the `Int` out of it. Furthermore, *we don't care what* `S` *actually is* in that case because we're going to throw it away. The action might as well be polymorphic in `S`.

In order to represent this, we'll introduce a new trait that represents `ST` actions that are safe to run—in other words, actions that are polymorphic in `S`:

```
trait RunnableST[A] {
  def apply[S]: ST[S,A]
}
```

This is similar to the idea behind the `Translate` trait from chapter 13. A value of type `RunnableST[A]` is a function that takes a *type* `S` and produces a *value* of type `ST[S,A]`.

In the previous section, we arbitrarily chose `Nothing` as our `S` type. Let's instead wrap it in `RunnableST` making it polymorphic in `S`. Then we don't have to choose the type `S` at all. It will be supplied by whatever calls `apply`:

```
val p = new RunnableST[(Int, Int)] {
  def apply[S] = for {
    r1 <- STRef(1)
    r2 <- STRef(2)
    x  <- r1.read
    y  <- r2.read
    _  <- r1.write(y+1)
    _  <- r2.write(x+1)
    a  <- r1.read
    b  <- r2.read
  } yield (a,b)
}
```

We're now ready to write the `runST` function that will call `apply` on any polymorphic `RunnableST` by arbitrarily choosing a type for `S`. Since the `RunnableST` action is polymorphic in `S`, it's guaranteed to not make use of the value that gets passed in. So it's actually completely safe to pass `()`, the value of type `Unit`!

The `runST` function must go on the `ST` companion object. Since `run` is protected on the `ST` trait, it's accessible from the companion object but nowhere else:

```
object ST {
  def apply[S,A](a: => A) = {
    lazy val memo = a
    new ST[S,A] {
      def run(s: S) = (memo, s)
    }
  }
  def runST[A](st: RunnableST[A]): A =
    st.apply[Unit].run(()).._1
}
```

We can now run our trivial program `p` from earlier:

```
scala> val p = new RunnableST[(Int, Int)] {
     |   def apply[S] = for {
     |     r1 <- STRef(1)
```

```
     |     r2 <- STRef(2)
     |     x  <- r1.read
     |     y  <- r2.read
     |     _  <- r1.write(y+1)
     |     _  <- r2.write(x+1)
     |     a  <- r1.read
     |     b  <- r2.read
     |   } yield (a,b)
     | }
p: RunnableST[(Int, Int)] = $anon$1@e3a7d65

scala> val r = ST.runST(p)
r: (Int, Int) = (3,2)
```

The expression `runST(p)` uses mutable state internally, but it doesn't have any side effects. As far as any other expression is concerned, it's just a pair of integers like any other. It will always return the same pair of integers and it'll do nothing else.

But this isn't the most important part. Most importantly, we *cannot* run a program that tries to return a mutable reference. It's not possible to create a `RunnableST` that returns a naked `STRef`:

```
scala> new RunnableST[STRef[Nothing,Int]] {
     |   def apply[S] = STRef(1)
     | }
<console>:17: error: type mismatch;
 found   : ST[S,STRef[S,Int]]
 required: ST[S,STRef[Nothing,Int]]
               def apply[S] = STRef(1)
```

In this example, we arbitrarily chose `Nothing` just to illustrate the point. The point is that the type `S` is bound in the `apply` method, so when we say `new RunnableST`, that type isn't accessible.

Because an `STRef` is always tagged with the type `S` of the `ST` action that it lives in, it can never escape. And this is guaranteed by Scala's type system! As a corollary, the fact that you can't get an `STRef` out of an `ST` action guarantees that if you have an `STRef`, then you are inside of the `ST` action that created it, so it's always safe to mutate the reference.

### A note on the wildcard type

It's possible to bypass the type system in `runST` by using the *wildcard type*. If we pass it a `RunnableST[STRef[_,Int]]`, this will allow an `STRef` to escape:

```
scala> val ref = ST.runST(new RunnableST[STRef[_, Int]] {
     |   def apply[S] = for {
     |     r1 <- STRef(1)
     |   } yield r1
     | })
ref: STRef[_, Int] = STRef$$anonfun$apply$1$$anon$6@20e88a41
```